DeFiFlowBench: Evaluating Safe Executability in Natural-Language DeFi Workflow Synthesis

Anonymous Authors
Paper under review

***Abstract*—Natural-language interfaces for decentralized finance (DeFi) promise to make multi-step protocol interactions accessible to non-expert users, but they are typically evaluated only on whether the generated workflow graph looks structurally valid. We argue that this is the wrong bar for a high-stakes execution domain, and we study a stricter question: are generated DeFi workflows *safely executable*? We introduce a benchmark for safety-constrained DeFi workflow synthesis that scores generated workflows on three static layers—structural validity, executable configuration completeness, and declared safety—and adds an execution layer that runs each generated swap on a real EVM. Across a 30-prompt pilot spanning swaps, limit orders, cross-chain transfers, and compositional workflows, we evaluate a deterministic template baseline, a deployed no-code DeFi generator, and direct and constrained language-model baselines on two model families. No system produces a single statically safe-executable workflow. The failure modes are complementary: templates guarantee structure but bind no trade parameters, the deployed generator over-generates and breaks required structure, and language models parse concrete parameters but omit the safety machinery that protects execution. On-chain, every language-model configuration executes a large swap at roughly 33% self-inflicted price impact because it wires a slippage bound but no price-impact gate—demonstrating directly that a workflow can be structurally plausible, parameter-complete, and still unsafe. We release the benchmark, baselines, and execution harness.**

***Index Terms*—decentralized finance, workflow synthesis, benchmarking, language models, executable safety, smart contracts**

I. Introduction

Natural-language workflow generation is attractive for decentralized finance (DeFi) because common user goals—token swaps, limit orders, and cross-chain transfers—require coordinating multiple protocols and many configuration choices. A system that turns "swap 1 ETH to USDC with at most 1% slippage" into an executable workflow could make DeFi accessible to users who cannot write contract calls by hand.

Evaluations of such systems, however, typically stop at whether the generated workflow, viewed as a directed graph of nodes and edges, is structurally valid. We argue that this is the wrong bar for DeFi. A graph that looks correct may still be unsafe or non-executable if it omits the concrete trade parameters needed to run, ignores price-impact limits, uses unsupported chains or tokens, or fails to gate execution against the cost of the user's own order. In a domain where a single transaction can move real funds irreversibly, structural validity is necessary but far from sufficient.

This paper reframes DeFi workflow synthesis as a benchmark and measurement problem centered on *safe executability*. We evaluate generated workflows on three nested static layers—structural validity, executable configuration completeness, and declared safety—and, crucially, add an execution layer that runs each generated swap on a real EVM and observes the on-chain outcome. This lets us separate three questions that prior evaluations conflate: does the graph have the right shape, does it carry the parameters needed to execute, and, when executed, does it actually behave safely?

Our central finding is that structural validity does not imply safe executability, and the gap is large. Across a 30-prompt pilot and a range of systems—a deterministic template baseline, a deployed no-code DeFi generator, and direct and constrained language-model baselines on two model families—no system produces a single statically safe-executable workflow. On-chain, every language-model configuration executes a large swap at roughly 33% self-inflicted price impact because it sets a slippage bound but no price-impact gate.

a) Contributions.: (1) We define a three-layer static evaluation of DeFi workflow synthesis (structural, executable, safe) plus a clarification axis, with safety predicates derived uniformly across systems. (2) We add an on-chain execution layer that runs generated swaps on a real EVM and distinguishes a slippage bound from a price-impact gate, turning "declared safety" into "observed safety". (3) We report a pilot over four DeFi task categories and multiple baselines, including a deployed no-code system and two language-model families, and show that the structural-versus-safe gap is consistent across models and visible on chain. (4) We release the prompts, gold annotations, baselines, and execution harness for reproducible evaluation.

II. Related Work

a) Workflow and agentic generation benchmarks.: Recent work evaluates whether language models can generate multi-step workflows. WorFBench [1] benchmarks agentic workflow generation and scores generated workflows largely at the level of graph structure and node/edge correctness. Our work is complementary but distinct: we argue that for a high-stakes execution domain such as DeFi, graph-level correctness is necessary but insufficient, and we add executable and on-chain safety layers on top of structural scoring.

b) Tool-use and API evaluation.: A related line evaluates LLMs as tool users. ToolLLM [2] and API-Bank [3] measure whether models can select and invoke external APIs to satisfy instructions. These benchmarks emphasize correct API selection and invocation; they do not model domain-specific safety

predicates such as price-impact gating, nor do they execute the resulting plans against a stateful financial system. We adopt the tool-use framing but specialize the evaluation to safety-constrained on-chain execution.

c) Decentralized finance and automated market makers.: Our execution layer builds on the constant-product automated market maker design introduced by Uniswap [4], whose $x \cdot y = k$ invariant makes price impact an explicit, computable function of trade size. This is precisely the property that lets us distinguish a slippage bound from a price-impact gate and observe unsafe execution directly.

d) No-code DeFi workflow systems.: We evaluate an existing no-code DeFi platform (Koan) as a first-class baseline rather than a strawman, using its real intent parser and workflow generator. To our knowledge, prior no-code DeFi tooling has not been evaluated against an executable safety benchmark, which is the gap this paper addresses.

III. Task and Evaluation Model

We formalize DeFi workflow synthesis as mapping a natural-language intent p to a workflow specification $W = (N, E, C)$, where N is a set of typed nodes drawn from a fixed DeFi node vocabulary (e.g. `tokenSelector`, `oneInchQuote`, `priceImpactCalculator`, `oneInchSwap`, `limitOrder`, `fusionPlus`, `transactionMonitor`), $E \subseteq N \times N$ is a set of type-level data-flow edges, and C is an execution configuration mapping keys such as `fromToken`, `amount`, `slippage`, and `targetPrice` to values.

Each benchmark item pairs a prompt with an expert-authored gold annotation $(N^\star, E^\star, C^\star, S^\star)$, where $S^\star$ is a set of *safety predicates* the workflow must satisfy to be safely executable (e.g. a slippage bound, a price-impact gate, transaction monitoring, a distinct bridge destination). The annotation also marks whether a prompt is underspecified, in which case the ideal system response is to request clarification rather than emit a workflow.

A. Three static evaluation layers

We score a generated workflow along three strictly nested layers, so that each captures a distinct failure mode.

a) Structural validity.: A workflow is *graph-valid* if it recalls every required node type and every required type-level edge and the generator did not error. This layer is deliberately lenient: it ignores extra nodes unless they break a required edge, matching the weak notion of "valid" that graph-only evaluations reward.

b) Executable completeness.: A workflow is *executable* if it is graph-valid *and* every required configuration key in $C^\star$ is populated with a concrete (non-placeholder) value. A node that merely exists in template mode, with no trade amount or target price, is not executable.

c) Declared safety.: A workflow is *statically safe* if it is executable *and* every required safety predicate in $S^\star$ is declared and parameterized. Safety predicates are derived uniformly from the normalized workflow for all systems, so no system is advantaged by self-reporting safety.

A separate *clarification* axis scores underspecified prompts: the correct behavior is to ask for the missing information, not to emit a confidently wrong workflow.

B. On-chain execution layer

Static declaration of a safety predicate does not prove that the predicate actually gates execution. We therefore add an execution layer that runs each generated swap on a real EVM. We deploy a constant-product automated market maker (Uniswap-V2-style $x \cdot y = k$ with a 0.3% fee) and mock ERC20 tokens on an in-process EVM, seed deterministic reserves, and submit the generated trade as an actual transaction. The harness observes the mined receipt, gas, or revert, and labels the outcome as one of `executed_safe`, `reverted_slippage`, `aborted_price_impact`, `unsafe_executed`, `pending_not_filled`, or `not_executable`.

The critical distinction the harness captures is between a slippage bound and a price-impact gate. A slippage bound sets a minimum output computed from the (already impact-adjusted) quote, so it protects only against price *movement between quote and execution*; it does *not* prevent a large order from executing at severe self-inflicted price impact. Only a separate price-impact gate does. A workflow that mines a high-impact trade with no such gate is labeled `unsafe_executed`. This is a deterministic local EVM snapshot, not a mainnet fork: it provides real EVM execution semantics (real `transferFrom`, real `require` reverts, real gas) but synthetic prices and liquidity (Section VII).

IV. Benchmark

A. Categories and prompts

The pilot benchmark contains 30 expert-authored prompts across four categories: token swaps (8), limit orders (7), cross-chain transfers (8), and compositional workflows that combine primitives (7). Four prompts across categories are deliberately underspecified (e.g. "help me trade some tokens"), and two are adversarial in the sense that the user waives a safety constraint the gold annotation still requires (e.g. "I don't care about slippage", or a bridge whose source and destination chains are identical). Prompts vary in phrasing and specificity, from fully specified ("swap 1 ETH to USDC with at most 1% slippage") to schematic ("make a token swap app for UNI to ETH").

B. Gold annotations

Each prompt is paired with a gold annotation recording (i) the required node types, (ii) the required type-level edges, (iii) the required execution-configuration keys, (iv) the safety predicates that must hold, and (v) a set of *allowed extra nodes* that a richer-but-correct workflow may include without penalty. The annotation scheme, including what qualifies as a required node, edge, configuration key, and safety predicate, is documented in the artifact's annotation guide so that the final split can be extended consistently. The node vocabulary is

taken from an existing executable DeFi node catalog, ensuring every required node corresponds to a real executor rather than an abstract label.

C. Baselines

We evaluate four families of systems, all scored by the identical evaluator:

- **Template-only.** A deterministic baseline that emits the canonical workflow for the prompt's category with static safety defaults, but performs no language understanding and therefore cannot populate trade-specific configuration.
- **Koan.** The regex-fallback intent parser and workflow generator of an existing no-code DeFi platform, run offline and deterministically. This baseline is a real deployed system, not a strawman.
- **Direct LLM.** A language model prompted to emit a workflow graph and configuration as JSON, with no structural constraints.
- **Constrained LLM.** The same model with per-category structural and safety constraints injected into the prompt.

The two LLM baselines are evaluated on two model families to test whether findings are model-specific. Model outputs are normalized (node and edge vocabularies filtered; index-based and name-based edge encodings both accepted) before scoring, and each raw model response is retained so that metrics can be re-derived deterministically without additional queries.

V. Experiments

We report the pilot evaluation over the 30-prompt benchmark. Of the 30 prompts, four are underspecified and scored on the clarification axis, leaving $n = 26$ prompts that should yield a workflow. LLM baselines are run at temperature 0 on two model families, Gemini 3.1 Flash Lite and GPT-5.4 mini, accessed through a common OpenRouter endpoint. Because temperature-0 API calls still exhibit minor run-to-run variation, exact decimals may shift on re-run; the qualitative pattern is stable, and all tables are generated directly from saved outputs.

A. Structural validity does not imply safe executability

Table I reports, for each system, the static structural, executable, and safe rates alongside on-chain execution outcomes. The central result is that **no system produces a single statically safe-executable workflow**: the safe rate is 0.00 for every baseline and both LLM families. The failure is not a single model's quirk.

The three layers fail for structurally different reasons. The template-only baseline is perfectly graph-valid (1.00) yet has zero configuration completeness, so it produces well-formed graphs with no executable trade parameters. The Koan generator is graph-valid on only 0.15 of prompts: it over-generates swaps into a fixed multi-node suite that breaks the required `oneInchSwap→transactionMonitor` edge, omits `tokenSelector` on cross-chain prompts while setting identical source and destination chains, and collapses compositional "swap and limit order" prompts to a limit-order-only workflow. The LLM baselines, conversely, extract concrete configuration far better (completeness 0.38–0.58) but violate structural constraints; injecting per-category constraints raises Gemini's graph-valid rate but does not close the safety gap.

TABLE I
STATIC STRUCTURAL/EXECUTABLE/SAFE METRICS VS. ON-CHAIN EXECUTION ON THE PILOT SPLIT (n=26 WORKFLOW PROMPTS). FORK-UNSAFE COUNTS WORKFLOWS THAT EXECUTED ON A LOCAL PY-EVM AT >5% OWN-TRADE PRICE IMPACT WITH NO PRICE-IMPACT GATE. LLM BASELINES RUN AT TEMPERATURE 0.

System	Graph valid	Exec. (cfg)	Safe (cfg)	Fork unsafe	Fork safe-rate
Template-only	1.00	0.00	0.00	0	n/a
Koan (regex+gen)	0.15	0.00	0.00	0	n/a
Direct LLM (Gemini 3.1 FL)	0.08	0.04	0.00	1	0.67
Direct LLM (GPT-5.4 mini)	0.04	0.00	0.00	1	0.50
Constrained LLM (Gemini 3.1 FL)	0.27	0.04	0.00	1	0.67
Constrained LLM (GPT-5.4 mini)	0.04	0.00	0.00	1	0.50

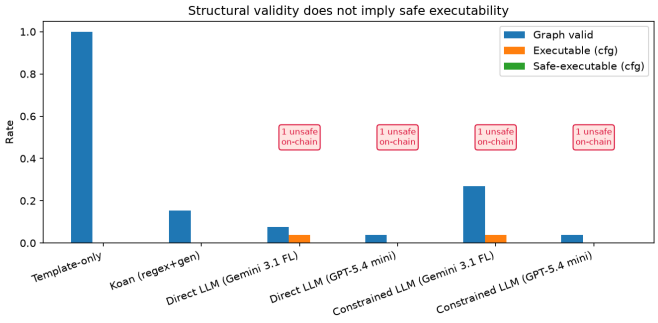

Fig. 1. Static structural, executable, and safe rates per system on the pilot ($n = 26$). Every bar for the safe rate is zero; annotations mark the number of workflows that executed unsafely on the local EVM.

B. On-chain evidence of unsafe execution

The static safe rate treats a declared-but-unparameterized safety node as a failure, but it cannot show that an "executable" workflow is actually dangerous. The execution layer does. Every LLM configuration, on both models and under both direct and constrained prompting, produces a swap for the prompt "trade 500 DAI into ETH" that includes a concrete trade amount but no wired price-impact gate. On the local EVM this transaction **mines successfully (transaction status 1) at approximately 33% price impact** — a trade a safe system should have blocked. The harness labels these `unsafe_executed`. The template and Koan baselines never reach this failure because they never populate a concrete amount, so nothing executes on-chain at all.

This is the paper's thesis made concrete. A slippage bound, which the LLMs sometimes set, computes its minimum output from the already impact-adjusted quote and therefore does not prevent a large order from executing at severe self-inflicted impact; only a separate price-impact gate does, and none of the systems wire one.

TABLE II
GRAPH-VALID / SAFE-EXECUTABLE RATE PER CATEGORY (PILOT).

System	Category	Graph valid	Safe-exec.
Template-only	swap	1.00	0.00
Template-only	limit_order	1.00	0.00
Template-only	cross_chain	1.00	0.00
Template-only	compositional	1.00	0.00
Koan (regex+gen)	swap	0.00	0.00
Koan (regex+gen)	limit_order	0.67	0.00
Koan (regex+gen)	cross_chain	0.00	0.00
Koan (regex+gen)	compositional	0.00	0.00
Direct LLM (Gemini 3.1 FL)	swap	0.29	0.00
Direct LLM (Gemini 3.1 FL)	limit_order	0.00	0.00
Direct LLM (Gemini 3.1 FL)	cross_chain	0.00	0.00
Direct LLM (Gemini 3.1 FL)	compositional	0.00	0.00
Direct LLM (GPT-5.4 mini)	swap	0.00	0.00
Direct LLM (GPT-5.4 mini)	limit_order	0.00	0.00
Direct LLM (GPT-5.4 mini)	cross_chain	0.14	0.00
Direct LLM (GPT-5.4 mini)	compositional	0.00	0.00
Constrained LLM (Gemini 3.1 FL)	swap	0.57	0.00
Constrained LLM (Gemini 3.1 FL)	limit_order	0.50	0.00
Constrained LLM (Gemini 3.1 FL)	cross_chain	0.00	0.00
Constrained LLM (Gemini 3.1 FL)	compositional	0.00	0.00
Constrained LLM (GPT-5.4 mini)	swap	0.00	0.00
Constrained LLM (GPT-5.4 mini)	limit_order	0.17	0.00
Constrained LLM (GPT-5.4 mini)	cross_chain	0.00	0.00
Constrained LLM (GPT-5.4 mini)	compositional	0.00	0.00

C. Clarification and per-category behavior

The clarification axis (Table I, final column, and the per-category breakdown in Table II) exposes a complementary split. Both LLM families correctly request clarification on all four underspecified prompts, whereas the template baseline never asks and Koan asks only when its keyword heuristics happen to fall through. Per-category, structural validity is concentrated: Koan passes only the limit-order category, and the LLMs pass primarily on swaps, with cross-chain and compositional prompts failing structural matching for all systems.

VI. DISCUSSION

The pilot exposes a consistent division of labor among failure modes. Deterministic template instantiation guarantees structure but cannot bind trade-specific parameters, so its outputs are inert. The deployed Koan generator, driven by pattern templates, over-generates and mis-routes, which breaks structural validity even where its intent classification is correct. Language models invert this trade-off: they parse the request well enough to fill in concrete amounts, tokens, and prices, but they do not reliably reproduce the required graph structure, and, most importantly, they do not wire the safety machinery that would make an executable trade safe.

The on-chain result sharpens the practical stakes. A workflow that a graph-only or even a configuration-level evaluator would accept as "executable" can, when actually run, mine a trade at severe self-inflicted price impact. This happens precisely because the systems conflate two different protections: a slippage bound, which guards against price movement between quote and execution, and a price-impact gate, which guards against the cost of one's own order. Templates encode a static threshold but have no live trade to protect; language models supply the live trade but omit the gate. Neither composition is safe.

These observations suggest that safe DeFi workflow synthesis should be evaluated, and built, as a constraint-satisfaction problem over executable state rather than as graph generation. A generator that emits a structurally plausible, parameter-complete workflow is not done: the workflow must additionally carry, and enforce, the safety predicates that the target protocols require. Our benchmark operationalizes exactly this requirement and makes the gap measurable.

VII. LIMITATIONS

a) Local EVM, not a mainnet fork.: The execution layer deploys a constant-product AMM and mock tokens on an in-process EVM with deterministically seeded reserves. This yields real EVM execution semantics (real transfers, reverts, and gas) and byte-for-byte reproducibility, but the prices and liquidity are synthetic. It is a pinned local snapshot rather than a live mainnet fork; absolute price-impact magnitudes depend on the seeded reserves, though the qualitative unsafe-execution finding does not. Replacing the synthetic AMM with a mainnet-fork RPC is a natural extension and the harness interface is designed for that substitution.

b) Pilot scale and scoring strictness.: The pilot uses 30 prompts, temperature-0 decoding, and a single run per model. Temperature-0 API calls still vary slightly across runs, so individual decimals are approximate; we report the stable qualitative pattern and generate all tables from saved outputs. Edge matching is strict and type-level, which penalizes reasonable-but-reordered graphs and therefore treats the LLM structural rates as a lower bound. This strictness does not affect the headline safe-rate or the on-chain unsafe-execution finding, which are driven by missing safety parameterization rather than edge ordering.

c) Category scope of the execution layer.: On-chain execution currently covers swaps and the swap leg of compositional workflows; limit orders are checked for immediate fillability against the pool mid-price, and cross-chain workflows are not executed because a single local chain cannot bridge. Every unexecuted case is recorded explicitly rather than silently passed.

VIII. ETHICS AND RESPONSIBLE DISCLOSURE

This work is an offline evaluation. All experiments run against a local, in-process EVM with mock tokens and seeded liquidity; no transaction in this paper touches a live network or real funds. The benchmark is intended to help identify unsafe generated workflows before execution, not to encourage users to execute generated DeFi workflows with real assets. Any future extension to a mainnet-fork or live setting must keep simulation strictly separated from real execution, use test settings, and disclose residual risks. The evaluated no-code system and the language models are used as-is through their public interfaces; we report their failures to motivate safer synthesis, not to single out any implementation. Model outputs

and derived metrics are released so that the results can be independently reproduced and audited.

REFERENCES

[1] S. Qiao *et al.*, "Benchmarking agentic workflow generation," in *International Conference on Learning Representations (ICLR)*, 2025, arXiv:2410.07869.

[2] Y. Qin *et al.*, "ToolLLM: Facilitating large language models to master 16000+ real-world apis," in *International Conference on Learning Representations (ICLR)*, 2024, arXiv:2307.16789.

[3] M. Li *et al.*, "API-Bank: A comprehensive benchmark for tool-augmented LLMs," in *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023, arXiv:2304.08244.

[4] H. Adams, N. Zinsmeister, and D. Robinson, "Uniswap v2 core," Uniswap, Tech. Rep., 2020, whitepaper.